Figure 26.3 The components of the ASP.NET Core logging framework. You register logging providers with an `ILoggerFactory`, which creates implementations of `ILogger`. You write logs to the `ILogger`, which delegates to the `ILogger` implementations that write logs to the console or a file. You can send logs to multiple locations with this design without having to configure the locations when you create a log message.

The design in figure 26.3 makes it easy to add or change where your application writes the log messages without having to change your application code. The following listing shows all the code required to add an `ILoggerProvider` that writes logs to the console.

Listing 26.1 Adding a console log provider in Program.cs

```
WebApplicationBuilder builder = WebApplication.CreateBuilder(args);
builder.Logging.AddConsole()                                          ❶

WebApplication app = builder.Build();

app.MapGet("/", () => "Hello World!");

app.Run();
```

❶ Adds a new provider using the Logging property on WebApplicationBuilder

NOTE The console logger is added by default by `WebApplicationBuilder`, as you'll see in section 26.3.

Other than this configuration on `WebApplicationBuilder`, you don't interact with `ILoggerProvider` instances directly. Instead, you write logs using an instance of `ILogger`, as you'll see in the next section.